Pandas

Pandas

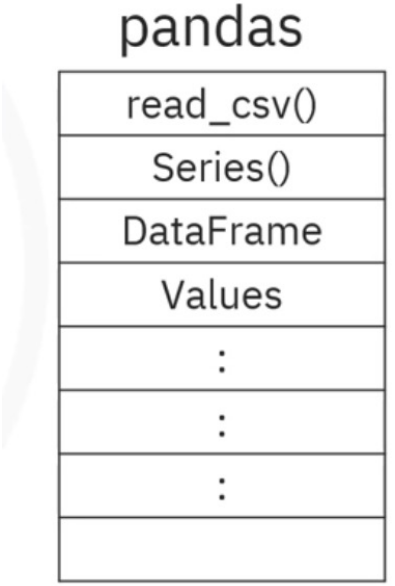
- Pandas is a Python library used for working with data sets.
- It has functions for analyzing, cleaning, exploring, and manipulating data.

Why Use Pandas?

- Pandas allows us to analyze big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.
- Relevant data is very important in data science.

What Can Pandas Do?

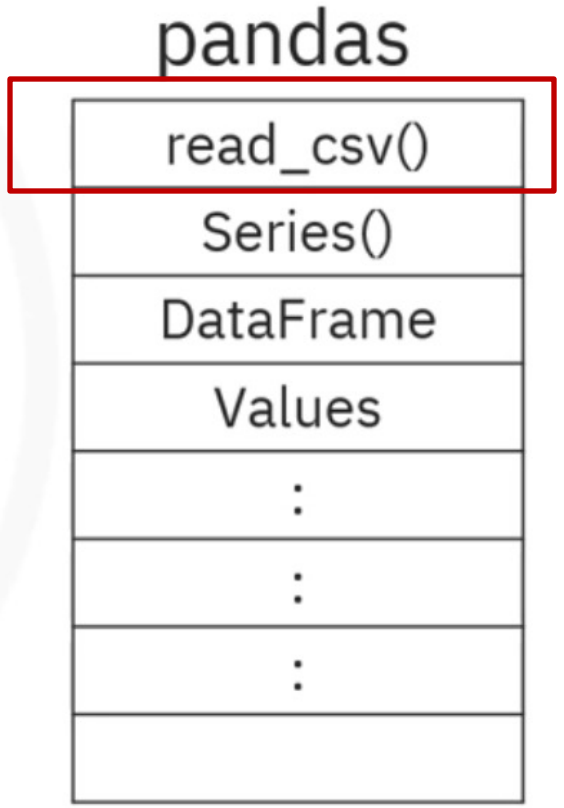
- Pandas gives you answers about the data. Like:
  - Is there a correlation between two or more columns?
  - What is average value?
  - Max value?
  - Min value?
- Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called *cleaning* the data.



```
import pandas

csv_path='file1.csv'

df=pandas.read_csv(csv_path)
```



- CSV, Excel files converted to dataframe

Pandas Data Structures

- **Data Structures**: Pandas offers two primary data structures
  - »DataFrame and Series

Pandas DataFrames

- A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.
- Pandas DataFrame is a two-dimensional, size-mutable, and heterogeneous data structure (similar to a table in a relational database or an Excel spreadsheet).

Pandas DataFrames

- Example:
- Create a simple Pandas DataFrame:
- import pandas as pd //Importing pandas library

```
data = {
  'calories': [420, 380, 390],
  'duration': [50, 40, 45]
}
```

Keys correspond to columns and values respond to rows

#load data into a DataFrame object:

df = pd.DataFrame(data) ← Creating a dataframe out of a dictionary

print(df)

	Calories	Duration
0	420	50
1	380	40
2	390	45

Example

- import pandas as pd

```
mydataset = {
  'cars': ["BMW", "Volvo", "Ford"],
  'passings': [3, 7, 2]
}

myvar = pd.DataFrame(mydataset)

print(myvar)
```

	Cars	Passings
0	BMW	3
1	Volvo	7
2	Ford	2

```
songs = {'Album' : ['Thriller','Back in Black', 'The Dark Side of the Moon',\ 'The Bodyguard','Bat Out of Hell'],
 'Released' : [1982,1980,1973,1992,1977],
'Length':['00:42:19','00:42:11','00:42:49','00:57:44','00:46:33']}
```

	Album	Length	Released
0	Thriller	00:42:19	1982
1	Back in Black	00:42:11	1980
2	The Dark Side of the Moon	00:42:49	1973
3	The Bodyguard	00:57:44	1992
4	Bat Out of Hell	00:46:33	1977

Dataframe with one column

- X = df[['Length']]

```
x=df[ ['Length'] ]
```

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

X

	Length
0	0:42:19
1	0:42:11
2	0:42:49
3	0:57:44
4	0:46:33
5	0:43:08
6	1:15:54
7	0:40:01

Dataframe with multiple columns

```
y=df[ ['Artist', 'Length', 'Genre '] ]
```

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

	Artist	Length	Genre
0	Michael Jackson	0:42:19	pop, rock, R&B
1	AC/DC	0:42:11	hard rock
2	Pink Floyd	0:42:49	progressive rock
3	Whitney Houston	0:57:44	R&B, soul, pop
4	Meat Loaf	0:46:33	hard rock, progressive rock
5	Eagles	0:43:08	rock, soft rock, folk rock
6	Bee Gees	1:15:54	disco
7	Fleetwood Mac	0:40:01	soft rock

Pandas DataFrames

- df.head() – First five rows of the dataframe
- df.tail() - View the last n rows of the DataFrame (default is 5 rows).
- df.info(): This method provides a concise summary of the DataFrame, including the number of non-null entries, column names, and data types.
- df.describe(): Returns description of the data in the DataFrame.
    - count - The number of not-empty values.
      mean - The average (mean) value.
      std - The standard deviation.
      min - the minimum value.
      25% - The 25% percentile*.
      50% - The 50% percentile*.
      75% - The 75% percentile*.
      max - the maximum value.

```
import pandas as pd
df = pd.read_csv('iris.csv')
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Pandas DataFrames

- Locate Row:
  - As you can see from the result above, the DataFrame is like a table with rows and columns.
  - Pandas use the loc attribute to return one or more specified row(s)

Pandas DataFrames

- Example
- Return row 0:
- #refer to the row index:
  print(df.loc[0])

Pandas Series

- A one-dimensional labeled array, essentially a single column or row of data.
- A Pandas Series is like a column in a table.
- It is a one-dimensional array holding data of any type.
- Example:
  - Create a simple Pandas Series from a list:
  - import pandas as pd

```
a = [1, 7, 2]

data1 = pd.Series(a)

print(data1)
```

Labels

- If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.
- This label can be used to access a specified value.
- With the index argument, you can name your own labels.

Labels

- Example
- Create your own labels:
- import pandas as pd

```
a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

	myVar
x	1
y	7
z	2

```
df_new=df
df_new.index=['a','b','c','d','e','f','g','h']
```

```
df.loc['a', 'Artist']:'Michael Jackson'
df.loc['b', 'Artist']:'AC/DC'
df.loc['a', 'Released']:1982
df.loc['b', 'Released']:1980
```

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
a	Michael Jackson	Thriller	1982	00:42:19	pop, rock, R&B	46.0	65	1982-11-30	NaN	10.0
b	AC/DC	Back in Black	1980	00:42:11	hard rock	26.1	50	1980-07-25	NaN	9.5
c	Pink Floyd	The Dark Side of the Moon	1973	00:42:49	progressive rock	24.2	45	1973-03-01	NaN	9.0
d	Whitney Houston	The Bodyguard	1992	00:57:44	R&B, soul, pop	27.4	44	1992-11-17	Y	8.5
e	Meat Loaf	Bat Out of Hell	1977	00:46:33	hard rock, progressive rock	20.6	43	1977-10-21	NaN	8.0
f	Eagles	Their Greatest Hits (1971-1975)	1976	00:43:08	rock, soft rock, folk rock	32.2	42	1976-02-17	NaN	7.5
g	Bee Gees	Saturday Night Fever	1977	01:15:54	disco	20.6	40	1977-11-15	Y	7.0
h	Fleetwood Mac	Rumours	1977	00:40:01	soft rock	27.9	40	1977-02-04	NaN	6.5

Df.loc[‘b’:’e’, ‘Artist’]

Navigating DataFrame – loc and iloc

- Both used for accessing rows and columns in dataframe

Label-based Indexing (loc)

- Access rows and columns by their labels
- Includes both the start and end positions in slicing
- df.loc[row_labels, column_labels]

Integer location-based Indexing (iloc)

- Access rows and columns by their integer positions
- Excludes end positions in slicing
- Does not accept Boolean data
- df.iloc[row_labels, column_labels]

```
df.loc[1, 'column_name']  # Accesses the row with label 1 and column 'column_name'
df.loc[1:3, 'col1':'col3']  # Accesses rows 1 to 3 and columns 'col1' to 'col3'
```

```
df.iloc[1, 0]  # Accesses the second row (index 1) and first column (index 0)
df.iloc[1:3, 0:2]  # Accesses rows at index 1 and 2, and columns at index 0 and 1
```

```
]: import pandas as pd
```

```
]: df = pd.DataFrame({'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]})
   mask = df['A'] > 2
   print(df.loc[mask])  # Will return rows where column 'A' has values greater than 2
```

```
   A  B
2  3  7
3  4  8
```

```
]: mask
```

```
]: 0    False
   1    False
   2     True
   3     True
   Name: A, dtype: bool
```

Navigating Data Frame

- iloc exclusively uses integer positions for accessing data.
- As a result, it makes it particularly useful when dealing with data where labels might be unknown or irrelevant.

- df.iloc[row number/slice]
- df.iloc[4], df.iloc[1:4], df.iloc[:],
- df.iloc[1:4, 5:8] --- SLICING

Dataframe Slicing

```
z=df.iloc[0:2, 0:3]
```

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

Z

	Artist	Album	Released
0	Michael Jackson	Thriller	1982
1	AC/DC	Back in Black	1980

Dataframe Slicing

```
z=df.loc[0:2, 'Artist':'Released']
```

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

z

	Artist	Album	Released
0	Michael Jackson	Thriller	1982
1	AC/DC	Back in Black	1980
2	Pink Floyd	The Dark Side of the Moon	1973

Navigating Data Frame

- df.iloc[4]-This command selects the 5th row (index 4) from the DataFrame df. It returns a single row as a Series.
- df.iloc[1:4]:This command selects a slice of rows from index 1 to 3 (excluding index 4) from df. It returns multiple rows as a DataFrame.

Navigating Data Frame

- df.iloc[:]
- -This command selects all rows and columns from df. It’s essentially the same as df, returning the entire DataFrame.
- df.iloc[1:4, 5:8]:This command selects rows from index 1 to 3 (excluding 4) and columns from index 5 to 7 (excluding 8). It returns the specified subset as a DataFrame.

Navigating Data Frame

- df.iloc[:,2]-This will select all rows (:) for the specified column index (3rd column), effectively giving you the entire column without specifically extracting any single row.
- This is the closest way to extract a column with .iloc without targeting individual rows.

Listing Unique Values

	Released
0	1982
1	1980
2	1973
3	1992
4	1977
5	1976
6	1977
7	1977

```
df['Released'] .unique()
```

create a new database consisting of songs from the 1980s and after

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

Applying inequality operators

df['Released']>1979

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	0:42:19	pop, rock, R&B	46.0	65	30-Nov-82	NaN	10.0
1	AC/DC	Back in Black	1980	0:42:11	hard rock	26.1	50	25-Jul-80	NaN	9.5
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49	progressive rock	24.2	45	01-Mar-73	NaN	9.0
3	Whitney Houston	The Bodyguard	1992	0:57:44	R&B, soul, pop	27.4	44	17-Nov-92	Y	8.5
4	Meat Loaf	Bat Out of Hell	1977	0:46:33	hard rock, progressive rock	20.6	43	21-Oct-77	NaN	8.0
5	Eagles	Their Greatest Hits (1971-1975)	1976	0:43:08	rock, soft rock, folk rock	32.2	42	17-Feb-76	NaN	7.5
6	Bee Gees	Saturday Night Fever	1977	1:15:54	disco	20.6	40	15-Nov-77	Y	7.0
7	Fleetwood Mac	Rumours	1977	0:40:01	soft rock	27.9	40	04-Feb-77	NaN	6.5

0	True
1	True
2	False
3	True
4	False
5	False
6	False
7	False

```
df1=df[df['Released’]>1979]
```

When you use the Boolean Series as an index (e.g., df[boolean_series]), pandas returns a new DataFrame containing only the rows where the Boolean Series has True values.

	Artist	Album	Released	Length	Genre	Music Recording Sales (millions)	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	Michael Jackson	Thriller	1982	00:42:19	pop, rock, R&B	46.0	65	1982-11-30	NaN	10.0
1	AC/DC	Back in Black	1980	00:42:11	hard rock	26.1	50	1980-07-25	NaN	9.5
3	Whitney Houston	The Bodyguard	1992	00:57:44	R&B, soul, pop	27.4	44	1992-11-17	Y	8.5

df1

Pandas Read CSV

- A simple way to store big data sets is to use CSV files (comma separated files).
- CSV files contains plain text and is a well know format that can be read by everyone including Pandas.
- In our examples we will be using a CSV file called 'data.csv'.

Pandas Read CSV

- Example:
- Load the CSV into a DataFrame:
- import pandas as pd

```
df = pd.read_csv('data.csv’)
df.head()      #show only first 5 rows
print(df.to_string()) #show all the rows
```

Pandas Read CSV

- The pd.read_csv() function is used to read the data from the data.csv file.
- df.to_string() converts the entire DataFrame df into a string representation, showing all rows and columns.
- If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:

Delete a column from Dataset

- You can delete a column or feature from a dataset-
  - **df.drop(labels, axis, index, columns, inplace)**

Method 1
Dropping either

  - **Axis = Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns')**
  - **Labels = Index or column labels to drop**
  - **df.drop(labels, axis, index, columns, inplace)**

Method 2
Dropping both

  - **Index = labels**
  - **Column = labels**

	Name	Age	Salary
0	Alice	24	50000
1	Bob	30	60000
2	Charlie	18	30000
3	David	35	80000

df.drop(labels=['Age'], axis=1, inplace=True)

	Name	Salary
0	Alice	50000
1	Bob	60000
2	Charlie	30000
3	David	80000

	Name	Age	Salary
0	Alice	24	50000
1	Bob	30	60000
2	Charlie	18	30000
3	David	35	80000

df.drop(index=[1], columns=['Salary'], inplace=True)

	Name	Age
0	Alice	24
2	Charlie	18
3	David	35

Delete a column from Dataset

– df.drop(df.columns[1], axis=1, inplace=True)

- **Column Selection**: df.columns[1] is used to select the second column.
- **Axis Parameter**: axis=1 specifies you are dropping a column. For rows, use axis=0.
- **Inplace=True -** If you want to modify the DataFrame in place.
- **Inplace=False-** If you do not want to modify the DataFrame in place

Delete a row from Dataset

- You can delete a row or feature from a dataset-
  - df.drop(1, axis=0, inplace=True)

- **Row Selection**: The first parameter '1' is used to select the first row.
- **Axis Parameter**: axis=0 specifies you are dropping a row.
- **Inplace=True -** If you want to modify the DataFrame in place.
- **Inplace=False-** If you do not want to modify the DataFrame in place